Problem 1 — Find the Promos: Solution

Repository: https://github.com/deep-rubiks003/promo-studio

The problem and the delivered product

An editor needs promo candidates across a long episodic show: each needs episode references, ordered start/end timestamps, an opening hook, an unresolved ending, a reason it works, and a posting order. Promo Studio turns timestamped TXT scripts into a reviewable source cut sheet. It gives instructions for an editor; it does not create finished video.

End-to-end approach

Upload episodes locally, choose a source workflow, preview the plan, generate source-backed cuts, inspect checks, compare repairs, optionally review quality and spoilers, then export. Chosen-episode mode supports pools and groups. Whole-show mode accounts for every uploaded episode and ranks one returned draft per episode; when a provider returns no usable draft, it states that gap rather than inventing one.

Architecture in plain language

Browser -> FastAPI -> local parser and source records -> bounded Gemini jobs -> deterministic checks -> human choice -> optional reviewer -> ranking and PDF. SQLite saves projects and job checkpoints. The provider worker is isolated so Stop can end local processing. The README contains the architecture diagram and module map.

How the output stays grounded

The model selects source ranges and event references. The application retrieves original dialogue from the source records. It checks source membership, structure, 20–60 second duration, dialogue boundaries and early hook evidence. Timecodes are screenplay estimates; the editor must confirm audio and footage. A boundary check cannot prove that a spoken thought is semantically complete.

Human-in-the-loop correction

The application identifies concrete local failures, feeds those failures and assigned source back into a repair request, and saves a proposal without replacing the original. A human sees both versions and chooses either. Multiple failed promos can be repaired sequentially. Even a still-failing proposal can be kept and reviewed: the override changes the chosen cut, not the truth of its failed checks. No returned draft means there is no new cut to accept.

Quality assessment and ranking

Virality Review is optional and explicit. A separate request assesses the assembled clip as a newcomer using Hook 25%, Standalone clarity 25%, Escalation 15%, Cliffhanger 25%, and Cut quality 10%. Real completed assessments produce a weighted score; unreviewed drafts remain unrated. This is editorial potential, not measured virality. Full-show ranking puts passed cuts first, then rated and unrated failed cuts, with reasons and all available source segments. Keeping a weak cut does not falsely mark it approved.

Efficiency and reliability

Uploads and plan previews have no model cost. Manual prompts use chosen sources. Whole-show work is split into bounded windows, processed sequentially and saved incrementally. Longer episodes can use multiple windows. Optional review and spoilers avoid paying for assessments the editor does not want. Saved stages support explicit resume; bounded retries handle temporary provider errors. The user chooses a spending limit, but provider quotas and unknown request billing still apply.

What failed and what changed

Quota exhaustion previously obscured completed work: durable checkpoints and partial output now preserve it. Repair requests previously appeared ineffective when ranking retained the old draft: distinct saved proposals and human decisions now make the change explicit. Valid JSON could have the wrong shape: normalization and source checks catch it. A failed cut could not be accepted or scored: the latest workflow permits both, preserves warnings and retains completed reviews on reload. Reordering after a score could switch the viewed episode: the reviewed episode now remains selected.

Editor experience and handoff

The scan plan opens in a popup. Starting it opens ranked cuts and highlights a live episode tracker: amber processing, green source completion and red failed windows. Drafts show dialogue, story beats and unanswered questions. Spoiler checking is a separate button against assigned episodes. PDF exports include ordered cuts, exact source dialogue, reasons, checks and available assessments; ranked full-show exports retain posting order.

Development method and evidence

The build followed an iterative AI-assisted workflow: inspect the current behavior, define the requested change, patch narrowly, test source and state invariants, exercise the browser with a fake provider, and incorporate human feedback. The correction and review loops are product workflows, not proof of autonomous self-improvement. Latest regression result: 147 passing tests. Synthetic browser checks demonstrate review of flagged cuts in both modes; no new paid inference or audience-performance experiment was used for this update. A paid complete 100-episode creative benchmark is not claimed.

Scope and next steps

This local prototype meets the cut-discovery and editor-handoff workflow. Production use would benefit from authentication, storage isolation, model-version evaluation and real editor feedback against posting outcomes. It currently does not render video or guarantee that model suggestions are compelling. Human judgment remains the final publishing decision.